

m1

Three.js Interactive Terrain Sculpting Engine | Level Tool, Grid Resolution Switching, Wireframe Visualization & Boundary Constraints System

It enables real-time procedural terrain editing through raycasting-based interaction and dynamic vertex manipulation.

The system transforms a static plane geometry into an interactive world-building environment where users can sculpt terrain, adjust resolution, visualize topology, and enforce geometric constraints in real time.

<https://www.udemy.com/course/web-based-3d-terrain-industrial-printing-engine-mastery/>

2. Core Terrain Sculpting System

The sculpting engine is based on **vertex-level displacement logic**.

Key behavior:

- Mouse input is converted into a 3D ray via Raycaster
- Intersection point defines sculpt center
- Vertices within brush radius are modified
- A quadratic falloff function controls deformation intensity

This creates smooth, organic terrain shaping such as:

- Mountains

- Depressions

3. Level / Flatten Tool (Assignment Feature)

A specialized tool that overrides standard brush behavior.

Functionality:

When active:

- All vertices inside brush radius move toward a fixed Y value
- Instead of additive deformation, terrain is normalized to a plane level

Example behavior:

- Target height: Y = 10
- Creates flat plateaus or artificial construction surfaces
- Simulates bulldozer or terrain leveling systems

This introduces controlled terrain engineering instead of organic deformation.

4. Dynamic Grid Resolution System

The engine supports real-time geometry regeneration.

Modes:

Low Poly Mode

- 64 × 64 segments
- High performance
- Suitable for large-scale editing

High Poly Mode

- 256 × 256 segments
- High detail
- Suitable for precision sculpting

Behavior:

- Current geometry is disposed
- New PlaneGeometry is generated
- Scene is updated without reloading

5. Wireframe Visualization System

A toggle system allows users to switch wireframe rendering on/off.

Purpose:

- Visualize mesh topology
- Understand vertex distribution
- Observe deformation behavior
- Debug terrain structure

This is essential for understanding how triangular mesh structures behave under deformation.

6. Boundary Constraint System

To prevent uncontrolled deformation, vertex height values are clamped.

Rules:

- Minimum height: $Y = 0$
- Maximum height: $Y = 50$

This ensures:

- Stable terrain structures
- Prevents infinite digging
- Maintains simulation realism

It introduces constraint-based procedural modeling concepts.

7. Raycasting Interaction Model

Raycasting converts:

2D mouse coordinates → 3D world position

Process:

- Mouse position is normalized

- This point becomes the sculpting origin

This is critical for all interactive 3D systems.

8. Falloff-Based Terrain Deformation

The system uses a quadratic falloff function:

- Strong deformation at center
- Gradual decrease toward edges

Changing the exponent affects terrain shape:

- Linear → sharp, mechanical deformation
- Quadratic → smooth hills
- Higher powers → steep peaks with soft edges

This teaches mathematical control of procedural geometry.

9. Coordinate System Transformation

The geometry is rotated using:

- `geo.rotateX(-Math.PI / 2)`

This converts:

- Plane XY surface → horizontal terrain (XZ plane)

After rotation:

- Y-axis becomes elevation (height)

This is a standard practice in terrain engines to align mathematical grids with world space navigation systems.

10. Performance Optimization Considerations

Current system loops through all vertices during interaction.

Challenges:

Optimization strategies include:

- Spatial partitioning (bounding box filtering)
- Brush area pre-selection
- Vertex index caching
- Reduced iteration scope
- GPU-based deformation (advanced)

These strategies are essential for scalable WebGL applications.

11. Lighting and Normal Recalculation

After every deformation:

- `geo.computeVertexNormals()` must be executed

Without it:

- Lighting becomes incorrect
- Shadows break visually
- Terrain appears flat or faceted incorrectly

Normals define how light interacts with geometry surfaces, making them essential for realism.

12. Learning Objectives

By completing this module, students will be able to:

- Implement raycasting-based interaction systems in Three.js
- Manipulate BufferGeometry vertices in real time
- Design procedural terrain sculpting tools
- Apply mathematical falloff functions to geometry
- Build dynamic mesh resolution systems
- Implement wireframe visualization tools
- Enforce geometric boundary constraints
- Understand coordinate transformations in 3D space
- Optimize vertex-heavy WebGL systems for performance
- Manage real-time synchronization between UI and rendering engine

© 2026 Okan Kaplan – MIT License [Read full license](#)

